

Analysis of CPU-Level Optimizations in Graph-Based Search: A Boggle Solver Case Study

Harrison T Farrell

Software Engineer, Sydney, Australia.

22-April-2026

Abstract

This paper investigates the impact of hardware-aware software design on the performance of a graph-based search problem (Boggle). By transitioning from a standard software implementation to a hardware-optimized model, a $11.0\times$ throughput increase was achieved, reducing mean execution time from 11,571 ns to 1,051 ns per board.

This paper analyzes the efficacy of mitigating the "Memory Wall" through an arena allocation, reducing branch miss prediction penalties via bit-masking, and leveraging hardware-accelerated bit-wise instructions. The findings suggest that performance in modern CPUs is less a factor of algorithmic complexity and more a function of memory-subsystem and execution-pipeline harmony.

The source code can be found here: <https://github.com/Harrison-Farrell/Boggle>

1 The Boggle Solver

Boggle is played on a grid of letters where the objective is to find valid words formed by sequential, adjacent cells. To solve this programmatically, two main components are required: a fast dictionary lookup mechanism and a comprehensive grid traversal algorithm. A Trie (prefix tree) is utilized as an efficient data structure for the dictionary. It allows the solver to determine in $O(1)$ time per character whether a sequence of letters forms a valid prefix or a complete word. The grid traversal is implemented using Depth-First Search (DFS). Starting from each cell on the board, the DFS recursively explores all unvisited neighboring cells while simultaneously traversing down the Trie. If the Trie indicates that the current path of letters does not form a valid prefix, the DFS prunes that branch and backtracks. This early termination significantly reduces the overall search space.

2 The Bottleneck of a Search

Traditional approaches to solving search problems focus primarily on reducing algorithmic time complexity. However, in high-performance computing contexts, an algorithm with optimal theoretical complexity may still execute slowly if its memory access patterns and branching logic clash with the underlying CPU architecture. Modern processors rely on deep execution pipelines, speculative execution, and hierarchical cache systems to achieve high throughput. When an algorithm frequently stalls the CPU—due to cache misses from scattered memory allocations, pipeline flushes from unpredictable branches, or resource contention from dynamic heap allocations—the physical hardware becomes the primary bottleneck. Therefore, achieving maximum throughput requires transitioning from a purely abstract, software-only logic to a hardware-aware co-design. This necessitates optimizing for three primary hardware constraints:

- **Memory Latency:** Minimizing pointer-heavy structures to reduce cache misses.
- **Branch Prediction:** Flattening conditional paths to maintain pipeline momentum.
- **Resource Contention:** Eliminating dynamic heap allocations during the search phase.

3 Memory Cache Optimization

Modern CPU performance is heavily dominated by the latency difference between L1 cache (~ 1 ns) and Main Memory (~ 100 ns). Standard Trie implementations use heap-allocated nodes connected via pointers. This creates "pointer-chasing" scenarios where the CPU stalls while waiting for non-contiguous memory fetches. By utilizing a `std::vector`-based Arena, The Trie nodes are laid out contiguously. This enables the CPU's hardware pre-fetcher to anticipate memory needs and significantly improves cache-line utilization. Reducing cache misses is the first priority in CPU-bound tasks.

```
1  class Trie {
2  public:
3      std::vector<Node> arena; // Contiguous block of memory
4
5      int32_t createNode() {
6          arena.emplace_back();
7          return arena.size() - 1; // Return index instead of pointer
8      }
9  };
10
```

Listing 1: Vector-Based Arena Trie

4 Branch Prediction Optimization

CPUs rely on deep pipelines and speculative execution. A miss predicted branch can flush 10–20 pipeline stages, causing severe performance degradation. The baseline solver traverses the Trie by checking 26 child pointers for each character. This introduces frequent branches. Instead implementing a 32-bit `children_mask` for each node. Before accessing child data, the CPU performs a bit-wise AND.

This effectively replaces a potential branch-heavy loop with a single bit-wise operation and a conditional check that the CPU can more easily predict or execute speculatively. Because bit-wise operations map directly to fundamental hardware instructions, they execute in a single clock cycle within the Arithmetic Logic Unit (ALU) and entirely bypass the complex, stall-prone control flow associated with standard branching.

```
1 struct Node {
2     uint32_t children_mask = 0; // Compact mask for pipeline efficiency
3     bool is_end_of_word = false; // Packed boolean
4     std::array<int32_t, ALPHABET_SIZE> children; // Contiguous block
5
6     Node() { children.fill(-1); }
7 };
8
```

Listing 2: Hardware-Aware Trie Node Structure

5 Hardware Bit-wise Pruning

Modern instruction set architectures (ISAs) include dedicated units for bit manipulation (e.g., BMI1/BMI2 on x86). The search for neighbors cells on the grid was optimized by representing neighbors as a 32-bit mask. Instead of iterating through a list of potential indices, `std::countr_zero` is used. This maps directly to hardware instructions such as TZCNT. This allows the CPU to find the next valid grid index in a single cycle without branching. Combined with clearing the lowest set bit (`bit-mask & (bit-mask - 1)`), the loop becomes a sequence of fast, predictable bit-wise instructions.

```
1 uint32_t bitmask = ADJACENT_CELLS[current_grid_index] & INDEX_MASK;
2 while (bitmask > 0) {
3     const uint32_t next_index = std::countr_zero(bitmask); // Hardware TZCNT
4     if (!(visited_mask & (1 << next_index))) {
5         // Branchless pruning via visit_mask bitwise checks
6         self(next_index, visited_mask, next_node);
7     }
8     bitmask &= (bitmask - 1); // BLSR instruction equivalent
9 }
10
```

Listing 3: Bit-Parallel Neighbor Traversal

6 Conclusion

The knowledge gained from this project underscores that for high-performance C++, software must be designed as a direct extension of the CPU’s architecture. By systematically addressing hardware bottlenecks—aligning data structures with cache lines, minimizing branch variance, and utilizing hardware-accelerated bit manipulation—an order-of-magnitude performance improvement was achieved. The substantial throughput increase demonstrates that when the execution pipeline and memory subsystem are kept fed and unobstructed, the underlying silicon can process data at theoretical peaks.

Furthermore, this study highlights a crucial paradigm shift in performance engineering. It suggests that "optimal" code is not solely defined by theoretical algorithmic complexity, often expressed in Big-O notation. While abstract models provide necessary foundational logic, true efficiency in modern systems demands an acute awareness of the physical hardware. Success relies heavily on maximizing the instructions-per-cycle (IPC) and minimizing the frequency of memory stalls, proving that mechanical sympathy is indispensable in modern optimization.

Optimization Layer	Time (ns)	Speedup	CPU Insight
Baseline Implementation	11,571	1.0×	Bound by Cache Misses
Vector Arena	9,789	1.18×	Improved L1/L2 Locality
Precomputed Neighbors	3,985	2.90×	Reduced Cycle Count
Bitmask Children/Neighbors	1,051	11.01×	Pipeline Harmony

Table 1: Systematic Throughput Improvement